RuFaS

A fazenda leiteira como problema de otimização — e como o RuFaS a resolve

Instalação, onde setar as variáveis do animal, execução e o custo da dieta

Pós-graduação em Inteligência Artificial · Prof. Dr. Leandro Alexandre Freitas · IFG

O desafio real: a vaca e a fazenda

Antes do código, entender o que estamos modelando.

A vaca tem exigências

Imagine uma vaca com cerca de 150 dias de lactação. Para viver e produzir leite, ela precisa de uma quantidade diária de energia, proteína, cálcio, fósforo e fibra. Essas exigências são calculadas por sistemas científicos (NRC e NASEM — padrões de exigência nutricional de bovinos) a partir do peso do animal, da fase de lactação e da produção de leite.

A fazenda é um sistema

A vaca faz parte de um rebanho, dentro de uma fazenda com solo, cultura, dejetos, clima e economia, todos interagindo. O RuFaS modela essa fazenda inteira, processo a processo.

A pergunta econômica

Muitas dietas atendem às exigências; elas custam valores diferentes.

Existem muitas combinações de ingredientes que atendem às exigências da vaca. Elas custam valores diferentes. A pergunta do produtor é direta:

Qual é a dieta mais barata que ainda atende a todas as exigências do animal?

Nutrir demais

Desperdiça dinheiro — nutrientes a mais que a vaca não aproveita viram custo e, às vezes, poluição.

Nutrir de menos

Compromete a saúde e a produção de leite — sai mais caro no fim.

O ponto ótimo

O menor custo que ainda cobre todas as exigências. É o que um otimizador calcula.

Os ingredientes e seu valor nutritivo

Cada alimento entrega nutrientes diferentes, a preços diferentes.

A dieta é montada com ingredientes (capim, caroço de algodão, silagem de milho, farelo de soja, milho moído, mistura mineral). Cada um tem composição nutricional conhecida e preço por quilo. Dois grupos: **volumosos** (fibrosos, pouca energia — capim, silagem; dão fibra e saciedade) e **concentrados** (pouca fibra, muita energia ou proteína — milho, farelo de soja).

Ingrediente	Energia	Proteína	Fibra (FDN)*	Papel na dieta
Silagem de milho	Alta	Baixa	Média	Volumoso energético
Feno de gramínea (capim)	Baixa	Baixa	Alta	Volumoso / fibra
Caroço de algodão	Alta	Média	Alta	Energia + fibra + gordura
Farelo de soja	Média	Muito alta	Baixa	Fonte de proteína
Milho moído	Muito alta	Baixa	Muito baixa	Concentrado energético

*** FDN** *(Fibra em Detergente Neutro): medida da fibra total do alimento. Valores qualitativos acima, só para intuição; no RuFaS os números exatos vêm da biblioteca NASEM/NRC.*

O problema da dieta, formalizado

Três ingredientes de todo problema de otimização.

Variáveis de decisão

O que podemos escolher: quanto de cada ingrediente entra na dieta, em kg de MS (matéria seca: o alimento sem a água) por vaca/dia.

$$x_1, x_2, \ldots, x_n$$

Função objetivo

O que queremos minimizar: o custo total da dieta = soma de (preço × quantidade) de cada ingrediente.

$$\min \quad \sum (\text{preço}_i \cdot x_i)$$

Restrições

O que não pode ser violado: a dieta tem de fornecer pelo menos a exigência de energia, proteína, cálcio (Ca), fósforo (P) e fibra.

$$\sum (\text{nutriente}_i \cdot x_i) \geq \text{exigência}$$

Um problema clássico da otimização

O problema da dieta é um dos mais antigos da programação linear.

De onde vem

- Anos 1940: o economista George Stigler buscou a dieta humana mais barata que atendia a 9 nutrientes — resolveu "no braço", de forma aproximada.
- Logo depois, George Dantzig criou o método simplex e resolveu o mesmo problema de forma exata.
- Virou o exemplo clássico de programação linear (PL) — e segue sendo usado na nutrição animal até hoje.

Programação linear (PL)

Quando o objetivo e as restrições são todos lineares nas variáveis (somas de preço×quantidade, nutriente×quantidade), o problema é um PL. Tem propriedades ótimas:

- Solução ótima garantida, quando existe.
- Resolve rápido, mesmo com muitas variáveis.
- Ferramentas acessíveis: solver do Excel, PuLP, SciPy.
- É o tipo de otimizador que será construído na atividade.

Instalação do RuFaS

Link de Materiais RuFaS

Linux/Ubuntu. O ponto crítico é a versão do Python.

1 · Obter o código e preparar o Python

```
git clone --recurse-submodules https://github.com/RuminantFarmSystems/RuFaS
cd RuFaS

# RuFaS exige Python 3.12/3.13 — use pyenv se o sistema tiver outra versão
pyenv install 3.13   &&   pyenv local 3.13
python -m venv venv  &&   source venv/bin/activate
```

2 · Instalar as dependências

```
pip install .   # lê o pyproject.toml e instala tudo
```

Armadilhas comuns: Python 3.14 quebra o numpy; use 3.13 via pyenv. A rede do IFG bloqueia o pip (erro SSL) — use um hotspot. Falta de RAM mata a simulação em silêncio ("0 erros", mas nenhum CSV) — garanta 8 GB+.

Execução e a cadeia de arquivos

Um comando dispara tudo — e ele lê uma cadeia de JSONs.

Rodar a simulação:

```
source venv/bin/activate
python main.py
```

Saída: um CSV em output/CSVs/ (~880 MB, 3.322 colunas, 7 anos).

O que main.py lê, em cadeia:

1	**task_manager_metadata.json**	O cenário padrão a rodar (freestall).
2	**tasks/example_freestall_task.json**	A sequência de tarefas da simulação.
3	**metadata/..._metadata.json**	O arquivo-mestre: liga animal + dieta + config.
4	**data/animal/ · data/feed/ · data/config/**	Os dados que VOCÊ edita.

Onde setar: as variáveis do animal

Arquivo: input/data/animal/example_freestall_animal.json

Campo (JSON)	O que é	Ex.
`herd_information.cow_num`	Número de vacas no rebanho	100
`herd_information.breed`	Raça (HO = Holandesa)	"HO"
`herd_information.annual_milk_yield`	Produção de leite do rebanho (kg/ano)	null
`management_decisions.milk_fat_percent`	Gordura do leite (%)	4
`management_decisions.milk_protein_percent`	Proteína do leite (%)	3.2
`farm_level.bodyweight.mature_body_weight_avg`	Peso adulto médio (kg)	740.1
`ration.user_input`	Travar a receita (true) ou otimizar livre (false)	true
`ration.formulation_interval`	Intervalo de reformulação (dias)	30

Por vaca (peso, DEL = dias em lactação, parto): *ficam em animal_population.json, gerado pela inicialização do rebanho; não se digitam um a um.*

Onde setar: a dieta, os preços e a nutrição

Dois lugares — a sua dieta, e a biblioteca de composição.

A dieta e os preços

`input/data/feed/example_Midwest_feed.json`

- feeds[].purchased_feed_cost — o preço de cada ingrediente (US$/kg MS).
- rations[].feeds[].ration_percentage — o % de cada ingrediente na receita fixada.
- Para incluir o caroço de algodão, adicione-o aqui (se existir na biblioteca).

A biblioteca nutricional

`input/data/feed/NASEM_Comp_with_TDN_urea.csv`

A tabela científica com a composição de cada alimento — é a fonte dos "coeficientes" do otimizador. Campos por ingrediente:

- Proteína (CP), fibra (NDF/ADF = FDN/FDA), energia (TDN = NDT, nutrientes digestíveis totais; e DE, energia digestível), gordura (EE).
- calcium, phosphorus e demais minerais.
- NASEM (padrão do freestall) ou NRC — escolhido no config.

O otimizador embutido do RuFaS

O RuFaS não só reporta o custo — ele resolve a dieta ótima.

Dentro do RuFaS existe o **`RationOptimizer`** (RUFAS/biophysical/animal/ration/ration_optimizer.py). A cada reformulação da ração, ele resolve o problema da dieta de custo mínimo usando SLSQP — um método de programação não linear da biblioteca SciPy.

A função objetivo que ele minimiza (ration_optimizer.py:812):

```
def objective(decision_vector, ration_config):
    # minimizar o custo de todos os alimentos satisfazendo as restrições
    return float(sum(np.multiply(decision_vector, ration_config.price_list)))
```

Caso de referência: o custo mínimo

Execução real do RationOptimizer, padrão NASEM, modo livre.

A vaca simulada

740 kg · 150 dias em lactação · 35 kg de leite/dia · gordura 4% · proteína 3,2%

Preços usados (US$/kg MS)

Silagem de milho	US$ 0,16
Silagem de leguminosa	US$ 0,18
Feno de gramínea	US$ 0,21
Milho moído	US$ 0,19
Farelo de soja	US$ 0,41
Caroço de algodão	US$ 0,36
Mistura mineral	US$ 0,80

Estimativas de mercado (EUA), ajustáveis.

Custo mínimo da dieta

US$ 4,65

por vaca / dia · US$ 0,133 por kg de leite

Principais ingredientes da solução

A dieta ótima combina milho moído, silagem de leguminosa e mistura mineral como base, com os demais volumosos entrando conforme as restrições. *(kg exatos: ver execução.)*

Para saber mais

Documentação, código e base científica do RuFaS.

Repositório oficial (GitHub)

O código-fonte do RuFaS. Clonar com --recurse-submodules.

github.com/RuminantFarmSystems/RuFaS

Documentação (GitHub Pages)

Guia de instalação, estrutura do código, testes e uso.

ruminantfarmsystems.github.io/RuFaS

Playlist de onboarding (YouTube)

Série de vídeos que acompanham a instalação e o uso.

youtube.com/playlist (onboarding RuFaS)

Artigo científico (PMC)

RuFaS como plataforma para pesquisa em laticínios sustentáveis.

ncbi.nlm.nih.gov/pmc/articles/PMC12766618

Ordem prática: README → documentação → artigos (para fundamentar o "porquê" dos cálculos).